

PROJETO DE REDES - 2026.1



O programa especificado abaixo deverá ser implementado utilizando-se as linguagens Python no Windows ou no Linux. O trabalho será **em duplas** e deverá ser enviado pelo SIGAA até as **23:59h do dia 06/08/2026**.

O objetivo deste projeto é implementar, **na camada de aplicação**, um serviço de entrega confiável de mensagens levemente baseado no TCP, **com implementação de pipelining**. O programa a ser desenvolvido receberá do servidor uma lista de músicas que poderão ser solicitadas através de um id, e em seguida deve receber corretamente o arquivo a ser transferido do servidor. O servidor pode ser acessado através do endereço IP **52.67.245.39**. Na camada de transporte deve ser utilizado o protocolo **UDP**, e a porta de destino deve ser a **50000**.

O fluxo geral de troca de mensagens esperado é o abaixo:

1. O cliente envia uma mensagem de criação de conexão (CONN_REQ) para o servidor, sorteando o número inicial de identificação de mensagens (número de 32 bits referente ao número do primeiro byte sendo transportado no segmento, assim como ocorre no TCP). O servidor responde com um CONN_ACK informando seu próprio número de identificação inicial, confirmando a criação da conexão, e informando também o número da conexão (inteiro de 16 bits), além de uma string descrevendo a lista de músicas que o cliente poderá solicitar (identificadores de 1 a 4);
2. O cliente envia uma mensagem do tipo MUSIC_SELECT contendo o identificador da música escolhida. Em seguida, o servidor envia a música através de uma sequência de mensagens do tipo MUSIC_RESPONSE usando pipelining (tamanho de janela fixo em 50 segmentos, tamanho máximo de segmento de 1000 bytes). Após o envio da música, o servidor envia uma mensagem do tipo MUSIC_RESPONSE_CONCLUDED, finalizando o fim do envio da música e enviando novamente a string de músicas disponíveis no servidor. Cada uma dessas mensagens deve ser respondida por um ACK do cliente, o qual tem o comportamento do ACK do TCP – é um ACK cumulativo que indica o número do próximo byte esperado. É importante observar que o servidor irá perder, propositalmente, uma determinada porcentagem dos ACKs encaminhados pelo cliente, o que irá gerar o envio de mensagens duplicadas que precisarão ser tratadas pelo cliente;
3. Após transferir o conteúdo do arquivo, o cliente tanto pode solicitar uma nova música quanto pode encerrar o programa, selecionando a opção 0.

A única solução de comunicação aceitável é o socket puro (popularmente conhecido como socket de baixo nível), não sendo permitidas outras bibliotecas de comunicação em rede. Por fim, ressaltamos que **toda mensagem deve possuir pelo menos o cabeçalho de 7 bytes**, podendo ser seguida ou não de bytes de dados. No cabeçalho, os 8 primeiros bits determinam o tipo da mensagem, os 32 bits seguintes determinam o número da mensagem (ordenação de bytes “little endian”), e os últimos 16 bits do cabeçalho determinam o número da conexão (ordenação de bytes “little endian”).

FORMATO DAS MENSAGENS – REQUISIÇÕES E RESPOSTAS

As mensagens de requisição possíveis de serem enviadas de um cliente para o servidor são as abaixo:

- **CONN_REQ – Iniciar conexão**

Mensagem a ser enviada do cliente para o servidor para solicitar a criação de um número de conexão.

Obs.: O servidor só mantém uma conexão ativa por até 10 minutos. Após esse período, se a conexão não tiver sido encerrada através de uma requisição do cliente, ela será automaticamente encerrada pelo servidor. Além disso, o servidor também encerrará uma conexão se ele tentar sem sucesso enviar um segmento por 10 vezes consecutivas.

1 byte	4 bytes	2 bytes
tipo (0x01) CONN_REQ	num_msg (0x???????) Número inicial de mensagem sorteado pelo cliente	num_conexao (0x0000) Deve ser 0, não utilizado nesse tipo de mensagem

- **tipo:** mensagens de inicialização de conexão devem ter o tipo 0x01 (decimal 1);
- **num_msg:** em mensagens de solicitação de conexão, esse campo deve ser preenchido com um número sorteado de 32 bits que servirá como identificador inicial de bytes da sessão. Para evitar ter que lidar com um comportamento circular do número da mensagem, é aceitável que o número sorteado seja inferior à capacidade máxima dos 4 bytes para permitir uma “folga” de crescimento do número sem que seja necessário resetar esse número para 0. Por exemplo, podem ser sorteados os 3 bytes menos significativos e deixar o último byte “travado” em 0 no sorteio;
- **num_conexao:** em mensagens de solicitação de conexão, esse campo deve ser preenchido com 0x0000 (decimal 0).

Essa mensagem será respondida por um CONN_ACK contendo o número de mensagem 0000 (decimal 0) e o número da conexão (inteiro de 16 bits)

1 byte	4 bytes	2 bytes	? bytes
tipo (0x02) CONN_ACK	num_msg (0x???????) Número inicial de mensagem sorteado pelo servidor	num_conexao (0x????) Número de conexão definido pelo servidor	payload String do menu a ser exibida pelo usuário, terminada pelo byte 0x00

- **MUSIC_SELECT – Selecionar música**

Mensagem a ser enviada do cliente para o servidor para solicitar a transmissão de uma música

1 byte	4 bytes	2 bytes	1 byte
tipo (0x03) MUSIC_SELECT	num_msg (0x????????) Número de mensagem do cliente	num_conexao (0x????) Número de conexão definido pelo servidor	payload (0x01 ou 0x02 ou 0x03 ou 0x04) Número da música escolhida pelo usuário

- **MUSIC_RESPONSE – Segmento de transmissão da música**

Mensagem a ser enviada do servidor para o cliente contendo um pedaço do arquivo da música

1 byte	4 bytes	2 bytes	até 1000 bytes
tipo (0x04) MUSIC_RESPONSE	num_msg (0x????????) Número de mensagem do servidor	num_conexao (0x????) Número de conexão definido pelo servidor	payload Até 1000 bytes contendo um pedaço do arquivo

- **MUSIC_RESPONSE_CONCLUDED – Segmento de finalização de transmissão da música**

Mensagem a ser enviada do servidor para o cliente comunicando o fim da transmissão da música e retransmitindo a string do menu a ser exibida para o usuário

1 byte	4 bytes	2 bytes	? bytes
tipo (0x05) MUSIC_RESPONSE_CONCLUDED	num_msg (0x????????) Número de mensagem do servidor	num_conexao (0x????) Número de conexão definido pelo servidor	payload String do menu a ser exibida pelo usuário, terminada pelo byte 0x00

- **ACK – Mensagem que o cliente em resposta a MUSIC_RESPONSE e MUSIC_RESPONSE_CONCLUDED**

Mensagem a ser enviada do cliente para o servidor após o recebimento de uma mensagem do tipo MUSIC_RESPONSE ou MUSIC_RESPONSE_CONCLUDED informando o número do próximo byte aguardado pelo cliente (ack cumulativo).

1 byte	4 bytes	2 bytes
tipo (0x06) ACK	num_ack (0x????????) Número do próximo byte esperado pelo cliente (ack cumulativo)	num_conexao (0x????) Número da conexão definido pelo servidor

- **CONN_FIN – Mensagem para encerramento de uma conexão**

Mensagem a ser enviada do cliente para o servidor solicitando o encerramento de uma conexão. Em resposta, o servidor apenas encerrará a conexão, sem envio de ACK.

1 byte	4 bytes	2 bytes
tipo (0x07) CONN_FIN	num_msg (0x????????) Número de mensagem do cliente	num_conexao (0x????) Número da conexão definido pelo servidor

- **TOO_SHORT_ERR – Mensagem do servidor informando que recebeu uma mensagem muito curta**

Mensagem informativa a ser enviada do servidor para o cliente informando que recebeu uma mensagem muito curta (inferior aos 7 bytes mínimos para uma mensagem). **Não** demanda um ACK por parte do cliente.

1 byte	4 bytes	2 bytes
tipo (0x08) TOO_SHORT_ERR	num_msg (0x00000000) Número 0	num_conexao (0x0000) Número 0

- **TOO_LONG_ERR – Mensagem do servidor informando que recebeu uma mensagem muito longa**

Mensagem informativa a ser enviada do servidor para o cliente informando que recebeu uma mensagem muito longa (superior a 1007 bytes máximos para uma mensagem). **Não** demanda um ACK por parte do cliente.

1 byte	4 bytes	2 bytes

tipo (0x09) TOO_LONG_ERR	num_msg (0x00000000) Número 0	num_conexao (0x0000) Número 0
---------------------------------------	--	--

- **INVALID_CONN – Mensagem do servidor informando que recebeu uma mensagem com número de conexão inválido**

Mensagem informativa a ser enviada do servidor para o cliente informando que recebeu uma mensagem com número de conexão inválido (ex.: número de conexão que já foi encerrada). **Não** demanda um ACK por parte do cliente.

1 byte	4 bytes	2 bytes
tipo (0x0A) INVALID_CONN	num_msg (0x00000000) Número 0	num_conexao (0x????) Número da conexão considerada inválida

- **INVALID_MUSIC – Mensagem informando escolha de música inválida**

Mensagem informativa a ser enviada do servidor para o cliente informando que recebeu uma mensagem com número de música inválido (ex.: menor do que 1, maior do que 4 ou mensagem não contém número de música). **Não** demanda um ACK por parte do cliente.

1 byte	4 bytes	2 bytes
tipo (0x0B) INVALID_MUSIC	num_msg (0x00000000) Número 0	num_conexao (0x????) Número da conexão definido pelo servidor

- **INVALID_MSG – Mensagem informando recebimento de número de mensagem inválido**

Mensagem informativa a ser enviada do servidor para o cliente informando que recebeu uma mensagem com número de mensagem inválido, ou seja, um número de mensagem maior do que o esperado. **Não** demanda um ACK por parte do cliente.

1 byte	4 bytes	2 bytes
tipo (0x0C) INVALID_MSG	num_msg (0x????????) Número de mensagem do cliente que o servidor está esperando	num_conexao (0x????) Número da conexão definido pelo servidor

Sobre o tratamento de perda de mensagens: o servidor retransmite sempre o **primeiro segmento não confirmado** (o mais antigo) e que tenha tido o seu timeout estourado. O servidor usa um tempo de timeout fixo de 1 segundo. O servidor também irá retransmitir o primeiro segmento não confirmado caso receba do cliente um número de mensagem inferior ao número de mensagem esperado (nesse caso, o servidor assumirá que uma mensagem foi perdida).

Outras observações importantes:

- O programa deverá tratar todas as mensagens que são possíveis de serem recebidas pelo servidor, imprimindo no console eventuais mensagens de erro;
- Cada grupo deverá tomar medidas para garantir que o código-fonte possua boa legibilidade (comentários são cruciais nesse sentido) e que o programa seja minimamente eficiente, ou seja, o programa não deverá realizar ações claramente desnecessárias para a solução do problema;
- O código implementado deve ser original, não sendo permitidas cópias de códigos inteiros ou grandes trechos de códigos de outras fontes (incluindo inteligências artificiais generativas, como ChatGPT);
- Caso o grupo decida utilizar um repositório de código como GitHub ou GitLab, certificar-se de deixar o repositório **privado** durante todo o prazo de entrega. Um repositório público é essencialmente um código compartilhado, o que é proibido no contexto deste projeto. Se o código da sua equipe for identificado em um repositório público, será atribuída nota 0 ao seu projeto.

--- Boa sorte! ---